

Titulación: Grado en Ingeniería Informática, Ingeniería en Sistemas de Información e InfoAde
Curso: 2024-2025. Convocatoria Ordinaria de Junio
Asignatura: Bases de Datos Avanzadas – Laboratorio

Practica 2: Carga Masiva de Datos, Procesamiento y Optimización de Consultas

ALUMNO 1:

Nombre y Apellidos: Fernando Pérez Holguín

DNI: 09066309P

ALUMNO 2:

Nombre y Apellidos: Tiziano Biondo Rouco

DNI: 08134385K

Fecha: 11/03/2025

Profesor Responsable: Óscar Gutiérrez Blanco

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se calificará la asignatura como Suspenso – Cero.

Plazos

Tarea en Laboratorio: Semana 10 de marzo, Semana 17 de marzo, Semana 24 de marzo, semana 31 de marzo y 7 de abril.

Entrega de práctica: Día 20 de abril, domingo a las 23:59. Aula Virtual

Documento a entregar: Este mismo fichero con las respuestas a las cuestiones planteadas, el programa que genera los datos de carga de la base de datos, los ficheros de log de PostgreSQL de esta práctica. No se piden los ficheros de los datos en bruto de la base de datos. Se entregará en un ZIP comprimido llamado: **DNI'sdelosAlumnos_PECL2.zip**

AMBOS ALUMNOS DEBEN ENTREGAR EL FICHERO EN LA PLATAFORMA.

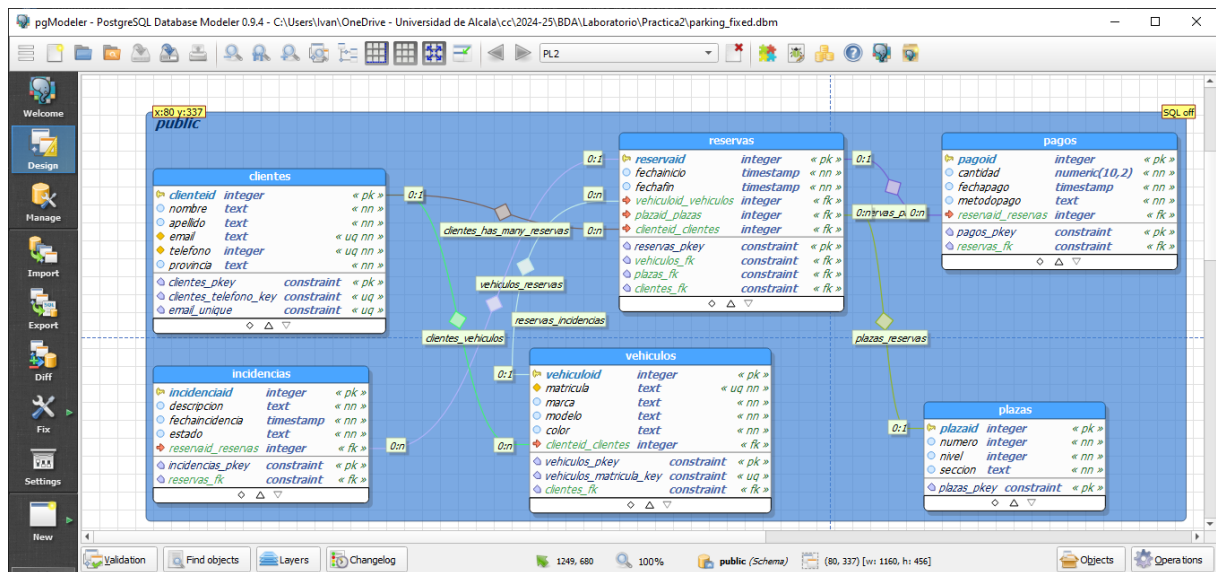
Introducción

El contenido de esta práctica versa sobre la monitorización de la base de datos, manipulación de datos, técnicas para una correcta gestión de los mismos, así como tareas de mantenimiento relacionadas con el acceso y gestión de los datos. También se trata el tema de procesamiento y optimización de consultas realizadas por PostgreSQL (17). Se analizará PostgreSQL en el proceso de carga masiva y optimización de consultas.

En general, la monitorización de la base de datos es de vital importancia para la correcta implantación de una base de datos, y se suele utilizar en distintos entornos:

- Depuración de aplicaciones: cuando se desarrollan aplicaciones empresariales no se suele acceder a la base de datos a bajo nivel, sino que se utilizan librerías de alto nivel y mapeadores ORM (Hibernate, Spring Data, MyBatis...) que se encargan de crear y ejecutar consultas para que el programador pueda realizar su trabajo más rápido. El problema en estos entornos está en que se pierde el control de qué están haciendo las librerías en la base de datos, cuántas consultas ejecutan, y con qué parámetros, por lo que la monitorización en estos entornos es vital para saber qué consultas se están realizando y poder optimizar la base de datos y los programas en función de los resultados obtenidos.
- Entornos de prueba y test de rendimiento: cuando una base de datos ha sido diseñada y se le cargan datos de prueba, una de las primeras tareas a realizar es probar que todos los datos que almacenan son consistentes y que las estructuras de datos dan un rendimiento adecuado a la carga esperada. Para ello se desarrollan programas que simulen la ejecución de aquellas consultas que se consideren de interés para evaluar el tiempo que le lleva a la base de datos devolver los resultados, de cara a buscar optimizaciones, tanto en la estructura de la base de datos como en las propias consultas a realizar.
- Monitorización pasiva/activa en producción: una vez la base de datos ha superado las pruebas y entra en producción, el principal trabajo del administrador de base de datos es mantener la monitorización pasiva de la base de datos. Mediante esta monitorización el administrador verifica que los parámetros de operación de la base de datos se mantienen dentro de lo esperado (pasivo), y en caso de que algún parámetro salga de estos parámetros ejecuta acciones correctoras (reactivo). Así mismo, el administrador puede evaluar nuevas maneras de acceso para mejorar aquellos procesos y tiempos de ejecución que, pese a estar dentro de los parámetros, muestren una desviación tal que puedan suponer un problema en el futuro (activo).

Para la realización de esta práctica será necesario generar una muestra de datos de cierta índole en cuanto a su volumen de datos. Para ello se generarán, dependiendo del modelo de datos suministrado, para una base de datos denominada **TELPARK**. Básicamente la base de datos guarda las reservas de los clientes que se realizan en el parking junto con su método de pago, los vehículos y las incidencias que se pueden producir. Se suministra el modelo relacional construido en **pgmodeler**.



Los datos referidos al año 2024 que hay que generar deben de ser los siguientes:

- Existen 3.000.000 clientes registrados en la APP de Telpark. Las provincias deben de ser asignada aleatoriamente de entre todas las provincias españolas.
- Existen 5.000.000 vehículos dados de alta. Se deben asignar los vehículos a los propietarios de manera aleatoria. Existen 500 marcas de automóviles diferentes donde cada una tiene entre 5 y 20 modelos, con colores entre 5 y 15 (el negro debe ser uno de ellos). La asignación de los valores de cada campo se debe de hacer de manera aleatoria.
- Existen 200.000 plazas de parking de la empresa Telpark. La planta debe ser un número aleatorio entre -10 y 0, y la sección un valor aleatorio comprendido entre las letras A-F.
- Se han producido 40.000.000 de reservas de plazas de parking por la APP. La fecha de inicio y final debe tener el día, mes, y hora generados de manera aleatorio, con una duración de estancia comprendida entre 1 día y 10 días.
- Cada reserva se ha pagado por un método de pago que puede ser efectivo, tarjeta de crédito, tarjeta de débito, PayPal, Bizzum o transferencia. La cantidad de la reserva debe ser el número de horas multiplicado por 3€. La fecha de pago debe ser anterior al comienzo de la estancia en el Parking.
- Se han generado 4.000.000 de incidencias generadas aleatoriamente de entre todas las reservas, donde el estado puede ser nueva, abierta, en proceso, resuelta y cerrada. Todas las asignaciones se hacen de manera aleatoria.

Actividades y Cuestiones

Cuestión 0: Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log ("line prefix") aparezca el DNI de los alumnos que realizan la

práctica (ambos), el nombre del host con su puerto, y la fecha y hora de la operación que se ha realizado. **Se ha de configurar también el servidor para que no use el procesamiento paralelo de consultas.**

Para realizar los cambios pedidos, hemos modificado el archivo C:\Program Files\PostgreSQL\16\data\postgresql.conf

Puesto que ya añadimos todas las configuraciones de la PL1 de esta asignatura, en esta ocasión solo hemos tenido que desactivar el procesamiento paralelo de consultas. Esto puede lograrse cambiando la línea 196 a `max_parallel_workers_per_gather = 0`

Cuestión 1: ¿Tiene el servidor postgres un recolector de estadísticas sobre los datos de cada campo de las tablas? Si es así, ¿Qué tipos de estadísticas se recolectan y dónde se guardan?

Sí, Postgres tiene un recolector de estadísticas que guarda información sobre datos de cada campo de las tablas. Dichos datos se guardan en `pg_statistics`, que a su vez contiene una vista para los usuarios llamada `pg_stats`. Algunos de los datos estadísticos que se guardan son:

- Número de columnas nulas
- Tamaño promedio de la columna
- El número estimado de valores distintos
- Los valores más comunes para la tabla y sus frecuencias
- Un histograma de los valores de la columna
- La correlación entre la ordenación física de las entradas en la tabla y los valores de la columna

Cuestión 2: Crear una nueva base de datos llamada **investigar** y que tenga las siguientes tablas con los siguientes campos y características:

- investigadores(codigo_investigador tipo numeric PRIMARY KEY, nombre tipo text, apellidos tipo text, salario tipo numeric)
- contratos(codigo_contrato tipo numeric PRIMARY KEY, nombre tipo text, entidad tipo text, coste tipo numeric)
- investigadores_contratos(codigo_investigador tipo numeric que sea FOREIGN KEY del campo codigo_investigador de la tabla investigadores con restricciones de tipo RESTRICT en sus operaciones, codigo_contrato tipo numeric que sea FOREIGN KEY del campo codigo_contrato de la tabla contratos con restricciones de tipo RESTRICT en sus operaciones, horas de tipo numeric. La PRIMARY KEY debe ser compuesta de codigo_investigador y codigo_contrato.

Se pide:

- Indicar el proceso seguido para generar esta base de datos.
- Cargar la información del fichero datos_empleados.csv, datos_proyectos.csv y datos_trabaja_proyectos.csv en dichas tablas de tal manera que sea lo más eficiente posible.
- Indicar los tiempos de carga.

Creamos la base de datos con “CREATE DATABASE investigar” y las tablas con el código presente en “PL2_c2.sql”.

Para cargar los datos de los csv a la base de datos de la forma más eficiente posible, hemos hecho lo siguiente:

- No utilizamos BEGIN ni COMMIT
- Usamos \COPY en vez de INSERT
- Añadimos las restricciones de FK después de los \COPY, en vez de hacerlos en el momento de crear la tabla
- Aumentamos temporalmente el tamaño de maintenance_work_mem, para acelerar el proceso de añadir las restricciones de FK
- Aumentamos temporalmente desde postgresql.conf el tamaño de max_wal_size para evitar un número demasiado elevado de checkpoints, que ralentizarían la carga
- Ponemos ANALYZE al final para que las estadísticas de las tablas estén actualizadas

Para medir de forma precisa los tiempos de carga, hemos activado el log de duración de forma temporal en postgresql.conf. Tras mirar el log, hemos determinado que los tiempos de carga son: 277 ms para la creación de la base de datos, 11, 2 y 5 ms para la creación y 9085, 594 y 76608 ms para la carga de las tablas investigadores, contratos e investigadores_contratos respectivamente, $4983 + 2814 = 7797$ ms para la inclusión de restricciones FK en investigadores_contratos, 1288 ms para el ANALYZE y <1ms para el set de maintenance_work_mem

Cuestión 3: Mostrar las estadísticas obtenidas en este momento para cada tabla. ¿Qué se almacena? ¿Son correctas? Si no son correctas, ¿cómo se pueden actualizar?

Las estadísticas de las tablas se almacenan en pg_stats y en pg_class

Los datos guardados son:

Información general: nombre del esquema, nombre de la tabla, nombre del atributo y si es heredado

Información de los datos: número de tuplas, fracción de las tuplas con valor nulo, longitud media de la columna, estimación de número de valores diferentes, lista de los valores (o de elementos si es una columna de arrays) más frecuentes con sus respectivas frecuencias, histograma de valores que dividen los valores en segmentos de tamaño parecido, correlación entre la ordenación física y lógica, histograma de repeticiones de elementos en columnas de arrays que dividen las repeticiones de elementos en segmentos de tamaño parecido, histograma de las longitudes no nulas de los valores, fracción de rangos vacíos de la columna e histograma de los las cotas superiores e inferiores de los rangos de la columna.

En caso de no ser correctas, las estadísticas pueden actualizarse utilizando el comando ANALYZE. Puesto que hemos incluido este comando después de la copia de datos, nuestras tablas sí poseen los datos correctos.

Cuestión 4: Aplicar el comando EXPLAIN a una consulta que obtenga la información de los investigadores con salario de menos de 24.000 euros. ¿Son correctos los resultados del comando EXPLAIN? ¿Por qué? Comparar con lo que se obtendría con lo visto en teoría obteniendo las estadísticas de las tablas con postgres.

Según los resultados de ejecutar el código de PL2_c4.sql, Postgres opta por hacer un escaneo secuencial de la tabla investigadores. Esto tiene sentido, pues el campo salario es un campo secundario no clave, y no existe ningún índice realizado sobre este. Por lo tanto, la elección de Postgres concuerda con la de teoría

Costes teóricos:

Secuencial -> $b_r = 28\ 040$

Índice -> No hay índice

Búsqueda binaria -> No está ordenado

Búsqueda secuencial con coste 28 040 es la única opción

Cuestión 5: Aplicar el comando EXPLAIN a una consulta que obtenga el número de contratos en los cuales el investigador trabaja 8 horas y tiene un salario de 20.000 euros. ¿Son correctos los resultados del comando EXPLAIN? ¿Por qué? Comparar con lo que se obtendría con lo visto en teoría obteniendo las estadísticas de las tablas con postgres.

Según los resultados de ejecutar el código de PL2_c5.sql, Postgres opta por hacer lo siguiente:

Primero, se realiza un escaneo secuencial de la tabla investigadores para obtener todos los registros cuyo salario sea igual a 20000.

A continuación, realiza una unión bucle anidado encauzando el resultado del escaneo anterior y comparándolo con la tabla investigadores_contratos. Se utiliza el índice investigadores_contratos_pkey para sacar más rápidamente las entradas con un código_investigador coincidente. Se aprovecha para comprobar si dichas entradas tienen un 8 en el campo horas.

Finalmente, se realiza una ordenación y agregación de la salida por la clave código_contrato. Esto se hace para asegurarse de que, si un contrato tiene varios investigadores que cumplen la condición, este solo se cuente una vez (gracias a DISTINCT).

La realización teórica puede encontrarse como anexo. Una diferencia a destacar es que en teoría no utilizaríamos el índice sobre código investigador de investigadores_contratos, pues ya habríamos leído la tabla para filtrar por las entradas con horas = 8. Sin embargo, parece que Postgres usa el índice investigadores_contratos_pkey en el join a la vez que filtra las entradas.

Tanto para este ejercicio como el siguiente, se han utilizado las estadísticas de Postgres (ver PL2_estadisticas_investigar.txt)

Cuestión 6: Aplicar el comando EXPLAIN a una consulta que obtenga la información del nombre de los contratos y entidad que tienen un coste mayor de 10000 y menor que 15000, y tienen investigadores de salario mayor de 16000 euros y trabajan 5 horas en ellos. ¿Son correctos los resultados del comando EXPLAIN? ¿Por qué? Comparar con lo que se obtendría con lo visto en teoría obteniendo las estadísticas de las tablas con postgres.

Según los resultados de ejecutar el código de PL2_c6.sql, Postgres opta por hacer lo siguiente: Primero, realiza un escaneo secuencial de la tabla contratos, comprobando que su coste esté entre los valores pedidos. A continuación, se crea una tabla hash sobre la clave codigo_contrato.

Después, se realiza otro escaneo secuencial, esta vez sobre la tabla investigadores_contratos con la condición de que el campo horas sea igual a 5. Una vez obtenidos ambos resultados, se realiza un hash join usando de condición la igualdad en el campo codigo_contrato, aprovechando la tabla hash creada para contratos en el paso anterior.

El resultado de la operación anterior pasa por un join realizado por bucle anidado con la tabla investigadores usando de condición la igualdad en el campo código_investigador. Se utiliza el índice investigadores_pkey para sacar más rápidamente las entradas con un código_investigador coincidente. Se aprovecha para comprobar si dichas entradas tienen un valor mayor que 16000 en el campo precio.

Finalmente, se realiza una ordenación y agregación de la salida por las claves nombre y entidad. Esto se hace para asegurarse de que, si un contrato tiene varios investigadores que cumplen la condición, este solo se cuente una vez (gracias a DISTINCT).

El árbol teórico puede encontrarse como anexo. En esta ocasión, Postgres realiza justo el mismo procedimiento que el que hemos aprendido en teoría

Cuestión 7: Generar los datos solicitados al comienzo de la práctica para la base de datos **TELPARK** creando un programa para tal fin que deberá de estar en un único fichero y comentado. Pegar el código del fichero en el cuadro de texto que se adjunta a continuación.

El código se encuentra en PL2_c7.sql

Cuestión 8: Realizar la carga masiva de los datos generados en la cuestión 7 en la base de datos **TELPARK**. Indicar el proceso seguido y el orden de carga de las tablas, explicando el porqué de dicho orden; **y asegurando la consistencia e integridad de los datos cargados**. Comparar los tiempos en las tablas implicadas y explicar a qué es debida la diferencia.

Tabla	Tiempo (seg)
vehiculos	47,874
clientes	44,921
plazas	0,583
reservas	170,154
incidencias	13,152
pagos	138,998

El orden de carga de las tablas no es importante. Las tablas vehiculos, reservas, incidencias y pagos tienen foreign keys que deben apuntar a tuplas existentes de la tabla correspondiente, pero para optimizar la carga de datos hemos añadido las restricciones de foreign key después de la carga de datos. Como en ese punto están todos los datos en sus tablas, el orden de carga y el orden de la agregación de las foreign key dan lo mismo.

Como las tablas no tienen restricciones, las variables principales que determinan cuánto tarda una tabla en cargar son el tamaño del archivo .csv y el tamaño de la tupla de la respectiva tabla. En este caso se puede observar que el orden de los tiempos de carga es muy similar al orden de los tamaños de los .csv

Tamaños de los .csv:

plazas < vehiculos $\approx$ clientes $\approx$ incidencias < pagos < incidencias

Tiempos de carga:

Plazas < incidencias < vehiculos $\approx$ clientes < pagos < incidencias

La única tabla que se desvía de lo esperado es incidencias, lo cual seguramente se debe a que el tamaño de la tupla de la tabla es más pequeño que el de las tuplas de vehiculos o clientes, las cuales tienen un .csv de aproximadamente el mismo tamaño que incidencias.

A partir de este momento en adelante, se deben de realizar las siguientes cuestiones con la base de datos que tiene la integridad referencial activada. Es obligatorio y queda prohibido cambiar la integridad referencial de la base de datos.

Cuestión 9: Realizar una consulta SQL que muestre el “porcentaje de clientes que cumplen que son de la comunidad de Andalucía realizando una reserva durante los meses de verano, pagando una cantidad de más de 150 €, estacionando sus vehículos por debajo de la planta -4, y que no hayan tenido incidencias cerradas con sus vehículos de color negro”.

Obtener el plan de ejecución con el resultado del comando EXPLAIN. Explicar la información obtenida en el plan de ejecución de PostgreSQL. Comparar el árbol obtenido por nosotros al traducir la consulta original al álgebra relacional y el que obtiene PostgreSQL. Comentar las posibles diferencias entre ambos árboles.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

La consulta SQL puede encontrarse en PL2_c9.sql

El tiempo de ejecución ha sido de 41.096ms para el EXPLAIN y de 132.309 s para la consulta en sí.

El resultado obtenido por la consulta ha sido 10.562066666666667%

Resultado comando EXPLAIN

La salida del EXPLAIN es demasiado grande como para entrar en este recuadro, por lo que se encuentra como anexo en PL2_c9_salida.txt

El orden seguido es el siguiente: primero se hace un escaneo secuencial de incidencias y un escaneo secuencial de reservas y se reúnen con un hash join. El resultado se reúne con el escaneo secuencial de vehiculos mediante otro hash join y el resultado espera a que se complete la otra parte del árbol de ejecución.

Por el otro lado se reúnen mediante hash joins los escaneos secuenciales de reservas, plazas, clientes y pagos; en ese orden. Se hacen las reuniones en orden de menor a mayor tamaño de las tablas reunidas.

Después, junta ambas ramas del árbol con un hash anti join y, por último, ordena el resultado y lo agrega para calcular el count(distinct [...]).

Cabe destacar que todas las selecciones se realizan con el escaneo secuencial inicial de cada tabla.

En el EXPLAIN, se puede apreciar que Postgres aplica automáticamente las reglas heurísticas para agilizar la consulta todo lo posible. Los múltiples filters intercalados demuestran que Postgres realiza las selecciones lo más pronto posible para reducir el número de tuplas, y los campos width indican que Postgres solo proyecta los campos necesarios para reducir el número de bloques. Además, los joins están ordenados para que las reuniones que produzcan el número de tuplas más pequeño salgan primero.

La estrategia de reunión utilizada es el Hash Join, que, dado que haya suficiente memoria, demuestra ser muy eficiente para entradas que no estén ordenadas ni dispongan de índices sobre la clave de reunión, especialmente si se encauzan dichas entradas.

Comentarios y explicaciones.

El árbol teórico se encuentra en PL2_c9_arbol.png

Hemos interpretado “y que no hayan tenido incidencias cerradas con sus vehículos de color negro” como que un cliente que tenga alguna incidencia cerrada con su vehículo negro en cualquiera de sus reservas no debe ser contado en el porcentaje.

La única diferencia apreciable entre el cálculo teórico y el resultado del comando EXPLAIN es el orden en que la reunión de reservas y plazas se reúne con clientes y con pagos. Postgres decide hacer la reunión con clientes primero porque, aunque el tamaño predicho de la reunión es igual, clientes es más pequeño que pagos. En cambio, en el árbol teórico decidimos hacer la reunión con pagos primero porque eso nos permite hacer una proyección que ignore el atributo reservasid antes y así reducir el tamaño de los registros.

Cuestión 10: Usando PostgreSQL, y a raíz de los resultados de la cuestión anterior, ¿qué acciones realizaría para mejorar el rendimiento de la consulta sin cambiar el código SQL y por qué? Obtener la información pedida de la cuestión 9 y explicar los resultados. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Comentar los resultados obtenidos y comparar con la cuestión anterior.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

1ª iteración: PL2_c10_1.sql (solo añadiendo más memoria):

Tiempo de ejecución: 9,453 ms (EXPLAIN), 68,945 s (consulta)

2ª iteración: PL2_c10_2.sql (lo anterior, pero añadiendo índices en todos los campos relacionados con la consulta: 13.633 ms (EXPLAIN), 61.678 s (consulta)

3ª iteración: PL2_c10_3.sql (lo anterior, pero ordenando físicamente las tablas por los índices creados anteriormente usando CLUSTER): 20.651 ms (EXPLAIN), 59.537s (consulta).

Resultado comando EXPLAIN

Los resultados de cada EXPLAIN se pueden observar en el anexo.

En la salida de la primera iteración, se puede apreciar que la estrategia tomada es la misma que en la cuestión 9.

En la salida de la segunda iteración, se puede observar que Postgres sustituye ciertos escaneos secuenciales por escaneos que utilizan algunos de los índices creados, con el objetivo de reducir el coste de la consulta, aunque a veces sigue utilizando el escaneo secuencial.

En la salida de la tercera iteración, podemos observar que, tras haber ordenado físicamente las tablas según el índice, Postgres utiliza el mismo plan de ejecución, pero el coste de las operaciones baja.

Comentarios y explicaciones.

De las 3 optimizaciones que hemos realizado, se ve que la más influyente ha sido la creación de índices sobre los campos relevantes para la consulta, seguida por la ordenación física mediante CLUSTER y, finalmente, el aumento de las especificaciones del sistema.

En la 2ª iteración, probamos tanto índices B como índices hash, pero obtuvimos mejores resultados con los índices B, aún si Postgres utilizaba frecuentemente los Hash Join antes de tener los índices

Cabe destacar que, si bien ordenar físicamente las tablas mediante CLUSTER permite reducir el coste, esta forma de optimización sería mucho más ineficiente si la base de datos se actualizase constantemente.

Aunque es cierto que esperábamos que el aumento de memoria fuese más efectivo, es cierto que ya mejoramos las especificaciones al principio de la práctica usando PGTune, por lo que es posible que el aumento realizado en la 1ª iteración no fuese tan relevante.

Cuestión 11: Usando PostgreSQL, borre el 30% de los clientes almacenados de manera aleatoria y todos sus datos relacionados de la manera más eficiente posible, ¿cuál ha sido el proceso seguido? ¿Y el tiempo empleado en el borrado? **Prohibido modificar la Integridad Referencial.**

Para realizar el borrado aleatorio del 30% de los clientes, hemos realizado los siguientes pasos (que pueden encontrarse en PL2_c11.sql):

1. Hemos aumentado la memoria disponible para realizar las operaciones de borrado
2. Hemos creado índices sobre los campos que se utilizan para determinar el borrado en cada una de las tablas (en unas, es clienteid o equivalente, en otras es reservaid o equivalente)
3. Creamos 2 tablas temporales: Una con el 30% de los clientes a borrar (escogidos de forma aleatoria con ORDER BY random), y otra con todas las reservas cuyo id de cliente se encontrase en la tabla anterior, llamada reservas_a_borrar
4. Creamos índices de clave primaria para las 2 tablas temporales mencionadas en el paso anterior
5. Borraremos las tablas pagos e incidencias cuyo campo reservaid_reservas apareciese en la tabla temporal, pues dependen de las reservas, pero ninguna otra tabla depende de ellas
6. Una vez liberadas las referencias a reservas, borramos de la tabla reservas todas las entradas que se encontrasen en la tabla temporal reservas_a_borrar
7. Una vez liberadas las referencias a vehículos que aparecían en la tabla reservas, se borran todas las entradas de vehículos cuyo campo clienteid_clientes aparezca en clientes_a_borrar
8. Una vez liberadas todas las referencias pertinentes, se borran todas las entradas de clientes cuya clave aparezca en clientes_a_borrar
9. Realizamos VACUUM FULL ANALYZE para hacer efectivo el ahorro de espacio y actualizar las estadísticas

Según el log, el tiempo total empleado en el borrado ha sido de $676+17+1284+146+152 = 2275$ s (unos 37 minutos)

Cuestión 12: Ejecute la consulta que pide los datos de la cuestión 9 de nuevo. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Comparar con los resultados anteriores.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

La consulta es la misma que la realizada en PL2_c9.sql

El porcentaje obtenido es algo mayor, un 10.5669523809523810%

El tiempo de ejecución es de 37.564ms (EXPLAIN) y 50.91s (consulta)

Resultado comando EXPLAIN

La salida del EXPLAIN puede encontrarse en PL2_c12_salida.txt

La estructura apenas ha cambiado desde la 4ª iteración del ejercicio 10. Ahora, la reunión entre clientes y pagos se hace por un bucle anidado con índices, en vez de con un hash join.

Puesto que el número de registros es menor, el coste y tiempo también han bajado.

Comentarios y explicaciones.

Parece que, gracias al VACUUM FULL ANALYZE, el proceso de borrado no ha afectado al desempeño de la consulta. Es más, gracias a que el número de registros es menor, los costes se han reducido, y Postgres ha optado por utilizar técnicas más favorables como la reunión por índice entre pagos y reservas en vez de leer secuencialmente y hacer un hash join

Cuestión 13: ¿Qué optimización/mejoras de la BD propondría para mejorar los resultados de dicho plan sin modificar el código SQL de la consulta? ¿Por qué? **No implementarlo ahora.**

Puesto que la base de datos mantiene todas las optimizaciones ya realizadas en el ejercicio 10, la única mejora relevante que se nos ocurre tras el borrado es ejecutar el comando VACUUM FULL ANALYZE para compactar las tablas y optimizar las lecturas de disco.

Sin embargo, ya implementamos esta mejora a la hora de hacer el ejercicio 11, por lo que el ejercicio 12 ya cuenta con la optimización.

Cuestión 14: Usando PostgreSQL, lleve a cabo las operaciones propuestas en la cuestión anterior y ejecute el plan de ejecución de la consulta que pide los datos de la cuestión 9. Obtener el plan de ejecución con el resultado del comando EXPLAIN. Compare los resultados del plan de ejecución con los de los apartados anteriores. Coméntelos.

Consulta SQL creada, tiempo de ejecución y resultado obtenido:

Respuesta idéntica al ejercicio 12 por las razones explicadas en el ejercicio 13

Resultado comando EXPLAIN

Respuesta idéntica al ejercicio 12 por las razones explicadas en el ejercicio 13

Comentarios y explicaciones.

Respuesta idéntica al ejercicio 12 por las razones explicadas en el ejercicio 13

Cuestión 15: A partir de lo visto y recopilado en toda la práctica. Describir y comentar cómo es el proceso de procesamiento y optimización que realiza PostgreSQL en las consultas del usuario.

Durante la realización de los ejercicios, hemos tenido múltiples ocasiones para observar algunos de los mecanismos de procesamiento y optimización de Postgres.

Varios de estos mecanismos concuerdan con los vistos en teoría, tales como la optimización heurística (reflejada en, entre otras cosas, el desplazamiento de las operaciones de selección, las proyecciones para descartar los atributos innecesarios), el uso de índices para leer de las tablas cuando sea conveniente, teniendo en cuenta la ordenación, la ordenación de las reuniones y la selección del algoritmo más apropiado para llevar estas a cabo en función de los recursos disponibles, etc.

Otros mecanismos nos han sorprendido, pues no los habíamos visto explícitamente en teoría. Este es el caso de filtrar por condición a la vez que se realiza un join, o el uso del anti join.

Para poder llevar a cabo todas estas funciones, Postgres ha hecho uso de las estadísticas que tiene sobre cada tabla y atributo. Prueba de ello es la mejora en el rendimiento tras usar ANALYZE.

Bibliografía

PostgreSQL (17)

- Capítulo 14: Performance Tips.
- Capítulo 19: Server Configuration.
- Capítulo 15: Parallel Query.
- Capítulo 24: Routine Database Maintenance Tasks.
- Capítulo 50: Overview of PostgreSQL Internals.
- Capítulo 68: How the Planner Uses Statistics.
- <https://pgtune.leopard.in.ua/>
- <https://explain.dalibo.com/>